

Pandas Series

```
In [9]: import pandas as pd  
import numpy as np
```

```
In [232... # In millions  
population = pd.Series([35.467, 63.951, 80.940, 60.665, 127.061, 64.511, 318.523])  
original_by_index = pd.Series([35.467, 63.951, 80.940, 60.665, 127.061, 64.511, 318
```

```
In [233... population
```

```
Out[233... 0      35.467  
1      63.951  
2      80.940  
3      60.665  
4     127.061  
5      64.511  
6     318.523  
dtype: float64
```

```
In [234... population.name = 'Population in millions'
```

```
In [235... population
```

```
Out[235... 0      35.467  
1      63.951  
2      80.940  
3      60.665  
4     127.061  
5      64.511  
6     318.523  
Name: Population in millions, dtype: float64
```

```
In [236... g7_pop.dtype
```

```
Out[236... dtype('float64')
```

```
In [237... g7_pop.values
```

```
Out[237... array([ 35.467,  63.951,  80.94 ,  60.665, 127.061,  64.511, 318.523])
```

```
In [238... type(population.values)
```

```
Out[238... numpy.ndarray
```

And they look like simple Python lists or Numpy Arrays. But they're actually more similar to Python dicts.

A Series has an index, that's similar to the automatic index assigned to Python's lists:

In [239... population[0]

Out[239... np.float64(35.467)

In [240... population.index

Out[240... RangeIndex(start=0, stop=7, step=1)

```
In [241... population.index = [                                # we can explicitly define the ind
    'Canada',
    'France',
    'Germany',
    'Italy',
    'Japan',
    'United Kingdom',
    'United States',
]
```

In [242... population

```
Out[242... Canada          35.467
France          63.951
Germany         80.940
Italy           60.665
Japan          127.061
United Kingdom  64.511
United States   318.523
Name: Population in millions, dtype: float64
```

We can say that Series look like "ordered dictionaries". We can actually create Series out of dictionaries:

Main

```
In [182... pd.Series({
    'Canada': 35.467,
    'France': 63.951,
    'Germany': 80.94,
    'Italy': 60.665,
    'Japan': 127.061,
    'United Kingdom': 64.511,
    'United States': 318.523
}, name='Population in millions')
```

```
Out[182... Canada          35.467
France          63.951
Germany         80.940
Italy           60.665
Japan          127.061
United Kingdom  64.511
United States   318.523
Name: Population in millions, dtype: float64
```

```
In [244... original_by_countryname = pd.Series(  
    [35.467, 63.951, 80.94, 60.665, 127.061, 64.511, 318.523],  
    index=['Canada', 'France', 'Germany', 'Italy', 'Japan', 'United Kingdom',  
          'United States'],  
    name='Population in millions')
```

```
In [245... pd.Series(population, index=['France', 'Germany', 'Italy', 'Spain'])
```

```
Out[245... France      63.951  
Germany      80.940  
Italy         60.665  
Spain         NaN  
Name: Population in millions, dtype: float64
```

Indexing

```
In [246... population['Japan']
```

```
Out[246... np.float64(127.061)
```

Numeric positions can also be used, with the `iloc` attribute:

```
In [247... population.iloc[0]
```

```
Out[247... np.float64(35.467)
```

Selecting multiple elements at once:

```
In [248... population[['Italy', 'France']]
```

```
Out[248... Italy      60.665  
France      63.951  
Name: Population in millions, dtype: float64
```

```
In [249... population.iloc[[0, 1]]
```

```
Out[249... Canada      35.467  
France      63.951  
Name: Population in millions, dtype: float64
```

```
In [250... population['Canada': 'Italy']
```

```
Out[250... Canada      35.467  
France      63.951  
Germany      80.940  
Italy         60.665  
Name: Population in millions, dtype: float64
```

Conditional selection (boolean arrays)

In [251... population

Out[251... Canada 35.467
 France 63.951
 Germany 80.940
 Italy 60.665
 Japan 127.061
 United Kingdom 64.511
 United States 318.523
 Name: Population in millions, dtype: float64

In [252... population > 64 # Boolean Condition

Out[252... Canada False
 France False
 Germany True
 Italy False
 Japan True
 United Kingdom True
 United States True
 Name: Population in millions, dtype: bool

In [253... population[population > 64] # Getting by condition, a filtering

Out[253... Germany 80.940
 Japan 127.061
 United Kingdom 64.511
 United States 318.523
 Name: Population in millions, dtype: float64

In [254... population.mean()

Out[254... np.float64(107.30257142857144)

In [255... population[population > population.mean()] # filtering

Out[255... Japan 127.061
 United States 318.523
 Name: Population in millions, dtype: float64

In [256... population.std()

Out[256... np.float64(97.24996987121581)

~ not ----- | or ----- & and

In [257... population[(population > population.mean() - population.std() / 2) | (population >

```
Out[257...] France          63.951
           Germany        80.940
           Italy          60.665
           Japan         127.061
           United Kingdom  64.511
           United States   318.523
Name: Population in millions, dtype: float64
```

Operations and methods

```
In [258...] population
```

```
Out[258...] Canada          35.467
           France          63.951
           Germany        80.940
           Italy          60.665
           Japan         127.061
           United Kingdom  64.511
           United States   318.523
Name: Population in millions, dtype: float64
```

```
In [259...] population * 1_000_000
```

```
Out[259...] Canada        35467000.0
           France        63951000.0
           Germany       80940000.0
           Italy         60665000.0
           Japan        127061000.0
           United Kingdom 64511000.0
           United States  318523000.0
Name: Population in millions, dtype: float64
```

```
In [260...] np.log(population)
```

```
Out[260...] Canada          3.568603
           France          4.158117
           Germany         4.393708
           Italy           4.105367
           Japan           4.844667
           United Kingdom   4.166836
           United States    5.763695
Name: Population in millions, dtype: float64
```

```
In [261...] population['France': 'Italy'].mean() # mean in sequential column
```

```
Out[261...] np.float64(68.518666666666666)
```

Modifying series

```
In [314...] population = original_by_countryname # avoiding mismatch when
```

```
In [315... population['Canada'] = 40.22
```

```
In [316... population.iloc[-1] = 800
```

```
In [317... population
```

```
Out[317... Canada          40.220
France            99.900
Germany           80.940
Italy             99.900
Japan            127.061
United Kingdom    99.900
United States     800.000
Name: Population in millions, dtype: float64
```

```
In [318... population[population < 70]
```

```
Out[318... Canada      40.22
Name: Population in millions, dtype: float64
```

```
In [322... population[population < 70] = 99.9          # CAUTION : this line "=" changes mul
```

```
In [320... population
```

```
Out[320... Canada          99.900
France            99.900
Germany           80.940
Italy             99.900
Japan            127.061
United Kingdom    99.900
United States     800.000
Name: Population in millions, dtype: float64
```

```
In [321... original
```

```
Out[321... 0      99.900
1      99.900
2      80.940
3      99.900
4     127.061
5      99.900
6     800.000
dtype: float64
```

CAUTION : "=" CAN CHANGE MULTIPLE VALUES PERMANENTLY

```
In [ ]:
```